

PROTOTYPE · ARCHITECTURE OVERVIEW

TTB Label Recognition

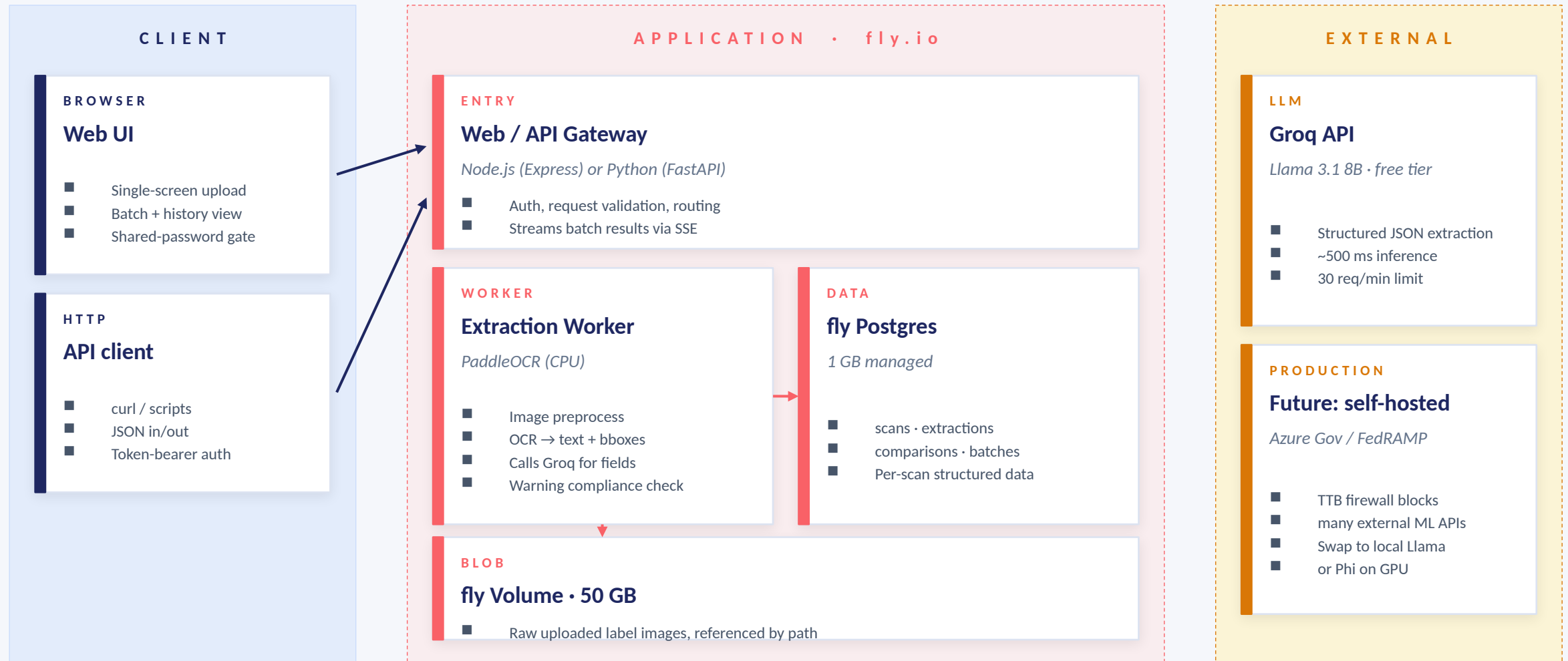
System Architecture

OCR + LLM pipeline · PaddleOCR · Groq Llama 3.1 8B

Deployed on fly.io · Web UI + documented API

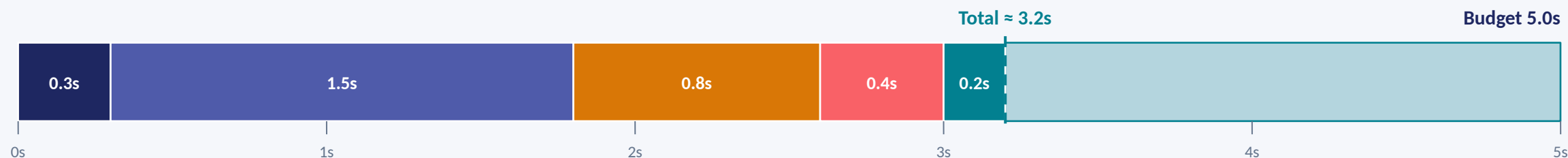
System Components

End-to-end view of the prototype — clients, fly.io services, external API



Single-Label Request — 5 s Latency Budget

Hard constraint per Sarah: prior vendor failed at 30–40 s. Current estimate: ~3.2 s with 1.8 s headroom.



Upload + auth

internal to fly.io

PaddleOCR (CPU)

internal to fly.io

Groq LLM extraction

external dependency

Warning check + DB

internal to fly.io

Render to UI

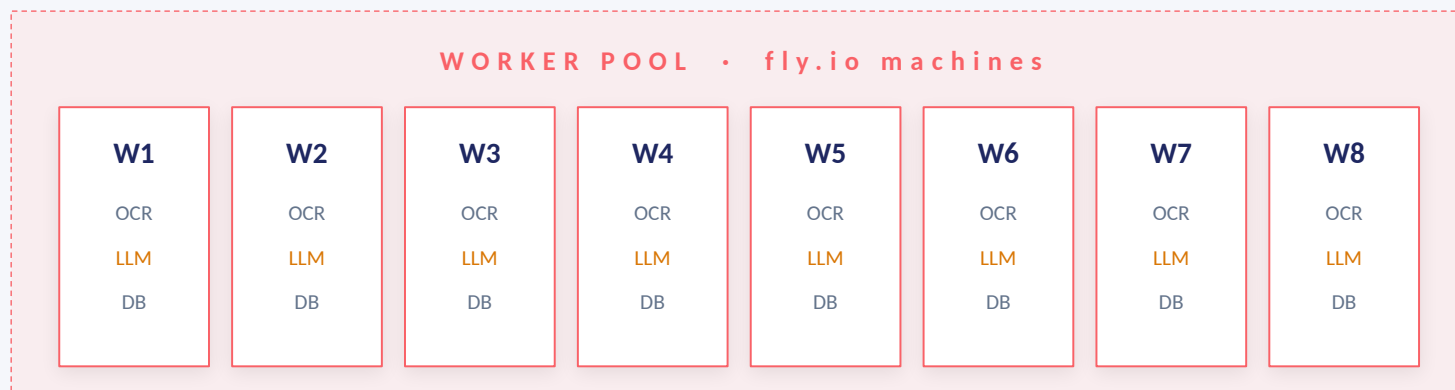
internal to fly.io

RISKS TO THE LATENCY BUDGET

- Groq cold-start or rate-limit retry can push the LLM step from 0.8 s to 2–3 s.
- PaddleOCR slows on large images; cap upload at ~10 MP and resize before OCR.
- Cold fly.io machine adds 1–2 s on first request; keep web machine always-on.

Batch Processing — 300 labels in ≤ 5 minutes

Sarah's peak-season ask: importers dump 200–300 applications at once. Parallel workers + streamed results.



THROUGHPUT MATH

300 labels $\times$ 3.2 s

$\div$ 8 workers

$\approx$ 2 minutes

Groq rate limit

Free tier $\approx$ 30 req/min. Worker concurrency capped by remaining tokens; degrade to $\sim$ 4 workers if needed.

Auto-scale

fly.io machines scale 1 $\rightarrow$ 8 based on queue depth; idle workers stop after 60 s to stay within $\sim$ \$20/mo.

Failure isolation

Per-label try/catch. One bad image marks that scan failed; the batch keeps going.

fly.io Deployment Topology

Single region near Groq · TLS at edge · shared-password gate · ~\$15–20/month at idle



Data Model

Four tables in fly Postgres. Images live on the fly volume and are referenced by path.

scans		
PK	id	<i>uuid</i>
FK	batch_id	<i>uuid?</i>
	application_id	<i>text?</i>
	beverage_type	<i>enum</i>
	image_path	<i>text</i>
	status	<i>enum</i>
	created_at	<i>timestampz</i>

comparisons		
PK	id	<i>uuid</i>
FK	scan_id	<i>uuid</i>
	reference_json	<i>jsonb</i>
	field_results	<i>jsonb</i>
	overall_status	<i>enum</i>
	confidence	<i>float</i>
	created_at	<i>timestampz</i>

extractions		
PK	id	<i>uuid</i>
FK	scan_id	<i>uuid</i>
	fields_json	<i>jsonb</i>
	ocr_raw_text	<i>text</i>
	warning_present	<i>bool</i>
	warning_exact	<i>bool</i>
	model_used	<i>text</i>

batches		
PK	id	<i>uuid</i>
	submitted_by	<i>text</i>
	label_count	<i>int</i>
	status	<i>enum</i>
	started_at	<i>timestampz</i>
	completed_at	<i>timestampz?</i>
	error_count	<i>int</i>

STORAGE STRATEGY

Postgres

- Structured data only
- JSONB for flexible extracted fields
- Daily fly snapshot in prototype

fly Volume

- Raw label images (JPG / PNG)
- Path stored in scans.image_path
- 50 GB ≈ 50 k images at 1 MB avg

Production swap → S3 / Azure Blob with KMS encryption + retention.

Prototype Today vs. Production Tomorrow

Everything on the right is acknowledged but explicitly deferred — tracked in REQUIREMENTS.md §8.

PROTOTYPE — WHAT WE'RE BUILDING

Hosting	fly.io single region (iad)
OCR	PaddleOCR in container
LLM	Groq Llama 3.1 8B (free tier)
Storage	fly Postgres + 50 GB volume
Auth	Shared password
Audit	Application logs only
Encryption	fly defaults (TLS + at-rest)
HA / DR	Single machine, daily snapshots
Observability	fly logs + /healthz
Cost	≈ \$15–20 / month

PRODUCTION — DEFERRED BACKLOG

Hosting	Azure Gov · FedRAMP region
OCR	Same, hardened pipeline + preprocess
LLM	Self-hosted Llama (firewall) on GPU
Storage	Azure Blob / S3 + KMS + retention
Auth	SSO · PIV-card · RBAC
Audit	Immutable audit log per NARA schedule
Encryption	FIPS 140-2 validated at-rest + in-transit
HA / DR	Multi-AZ Postgres · regional failover
Observability	APM, metrics, alerting, on-call rota
Cost	Procurement-driven, TBD